

DEPLOYMENT_GUIDE

HelixCode Production Deployment Guide

Overview

This guide provides comprehensive instructions for deploying HelixCode in production environments. HelixCode supports multiple deployment strategies including Docker containers, Kubernetes orchestration, and traditional server deployments.

Table of Contents

- [Prerequisites](#)
- [Quick Start](#)
- [Docker Deployment](#)
- [Kubernetes Deployment](#)
- [Cloud Platform Deployment](#)
- [Traditional Server Deployment](#)
- [High Availability Setup](#)
- [Monitoring and Observability](#)
- [Backup and Recovery](#)
- [Security Hardening](#)
- [Performance Tuning](#)
- [Troubleshooting](#)

Prerequisites

System Requirements

Minimum Requirements: - CPU: 4 cores - RAM: 8 GB - Storage: 50 GB SSD - Network: 100 Mbps

Recommended for Production: - CPU: 8+ cores - RAM: 16 GB+ - Storage: 100 GB+ SSD - Network: 1 Gbps

Software Dependencies

- **Go:** 1.24.0+ (for source deployments)
- **Docker:** 20.10+ (for containerized deployments)
- **PostgreSQL:** 13+ (primary database)
- **Redis:** 6+ (caching and sessions)
- **Nginx/HAProxy:** For load balancing (optional)

Network Requirements

- **Inbound Ports:**
 - 80/443: HTTP/HTTPS
 - 2222: SSH (for worker connections)

- 8080: API port (if not behind reverse proxy)
- **Outbound Access:**
 - LLM provider APIs (OpenAI, Anthropic, etc.)
 - Package repositories
 - NTP servers
 - DNS servers

Quick Start

Docker Compose (Recommended for Evaluation)

```
# Clone the repository
git clone https://github.com/your-org/helixcode.git
cd helixcode

# Configure environment
cp .env.example .env
# Edit .env with your API keys and database credentials

# Start all services
docker-compose up -d

# Check deployment status
docker-compose ps
curl http://localhost/health

# View logs
docker-compose logs -f helixcode
```

Single-Node Production Setup

```
# Install dependencies
sudo apt update
sudo apt install postgresql redis-server nginx

# Setup database
sudo -u postgres createdb helixcode_prod
sudo -u postgres createuser helixcode
sudo -u postgres psql -c "ALTER USER helixcode PASSWORD
    'secure_password';"

# Download and configure HelixCode
wget https://github.com/your-org/helixcode/releases/latest/
    download/helixcode-linux-amd64.tar.gz
tar -xzf helixcode-linux-amd64.tar.gz
sudo mv helixcode /usr/local/bin/

# Create configuration
sudo mkdir -p /etc/helixcode
```

```

sudo tee /etc/helixcode/config.yaml > /dev/null <<EOF
server:
  host: 0.0.0.0
  port: 8080
database:
  host: localhost
  port: 5432
  user: helixcode
  password: secure_password
  dbname: helixcode_prod
redis:
  host: localhost
  port: 6379
EOF

# Create systemd service
sudo tee /etc/systemd/system/helixcode.service > /dev/null <<EOF
[Unit]
Description=HelixCode AI Development Platform
After=network.target postgresql.service redis-server.service

[Service]
Type=simple
User=helixcode
Group=helixcode
ExecStart=/usr/local/bin/helixcode server --config /etc/
        helixcode/config.yaml
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

# Start service
sudo systemctl daemon-reload
sudo systemctl enable helixcode
sudo systemctl start helixcode

```

Docker Deployment

Single Container Setup

```

FROM golang:1.26-alpine AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

```

```
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o helixcode ./cmd/server

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/helixcode .
COPY --from=builder /app/config ./config

EXPOSE 8080
CMD ["/helixcode", "server", "--config", "./config/config.yaml"]
```

Docker Compose Production Setup

```
version: '3.8'
```

```
services:
```

```
  helixcode:
```

```
    image: helixcode/helixcode:latest
    container_name: helixcode-server
    restart: unless-stopped
    environment:
      - HELIXCODE_ENV=production
      - HELIXCODE_DATABASE_HOST=postgres
      - HELIXCODE_DATABASE_PASSWORD=${DB_PASSWORD}
      - HELIXCODE_REDIS_PASSWORD=${REDIS_PASSWORD}
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
```

```
    ports:
```

```
      - "8080:8080"
```

```
    volumes:
```

```
      - ./config:/app/config:ro
      - ./logs:/app/logs
```

```
    depends_on:
```

```
      - postgres
      - redis
```

```
    networks:
```

```
      - helixcode-network
```

```
  postgres:
```

```
    image: postgres:15-alpine
    container_name: helixcode-postgres
    restart: unless-stopped
    environment:
      - POSTGRES_DB=helixcode_prod
      - POSTGRES_USER=helixcode
      - POSTGRES_PASSWORD=${DB_PASSWORD}
```

```

volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql
networks:
  - helixcode-network

redis:
  image: redis:7-alpine
  container_name: helixcode-redis
  restart: unless-stopped
  command: redis-server --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis_data:/data
  networks:
    - helixcode-network

nginx:
  image: nginx:alpine
  container_name: helixcode-nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - ./ssl:/etc/nginx/ssl:ro
  depends_on:
    - helixcode
  networks:
    - helixcode-network

volumes:
  postgres_data:
  redis_data:

networks:
  helixcode-network:
    driver: bridge

```

Multi-Stage Docker Build

```

# Build stage
FROM golang:1.26-alpine AS builder

# Install build dependencies
RUN apk add --no-cache git ca-certificates tzdata

# Set working directory
WORKDIR /app

```

```
# Copy go mod files
COPY go.mod go.sum ./
RUN go mod download

# Copy source code
COPY . .

# Build the application
RUN CGO_ENABLED=0 GOOS=linux go build \
    -ldflags="-w -s -X main.version=${VERSION} -X \
        main.buildDate=$(date -u +'%Y-%m-%dT%H:%M:%SZ')" \
    -o helixcode ./cmd/server

# Runtime stage
FROM alpine:3.18

# Install runtime dependencies
RUN apk --no-cache add ca-certificates tzdata \
    && addgroup -g 1001 -S helixcode \
    && adduser -u 1001 -S helixcode -G helixcode

# Set working directory
WORKDIR /app

# Copy binary and config
COPY --from=builder /app/helixcode .
COPY --from=builder /app/config ./config

# Create necessary directories
RUN mkdir -p /app/logs /app/data \
    && chown -R helixcode:helixcode /app

# Switch to non-root user
USER helixcode

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s -- \
    retries=3 \
    CMD wget --no-verbose --tries=1 --spider http:// \
        localhost:8080/health || exit 1

# Expose port
EXPOSE 8080

# Start application
CMD ["/helixcode", "server", "--config", "./config/config.yaml"]
```

Kubernetes Deployment

Namespace and RBAC Setup

```
apiVersion: v1
kind: Namespace
metadata:
  name: helixcode
  labels:
    name: helixcode
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: helixcode-sa
  namespace: helixcode
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: helixcode-role
rules:
- apiGroups: [""]
  resources: ["pods", "services", "configmaps", "secrets"]
  verbs: ["get", "list", "watch", "create", "update", "patch",
    "delete"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets", "statefulsets"]
  verbs: ["get", "list", "watch", "create", "update", "patch",
    "delete"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: helixcode-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: helixcode-role
subjects:
- kind: ServiceAccount
  name: helixcode-sa
  namespace: helixcode
```

ConfigMap and Secrets

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: helixcode-config
  namespace: helixcode
data:
  config.yaml: |
    server:
      host: 0.0.0.0
      port: 8080
    database:
      host: helixcode-postgres
      port: 5432
      user: helixcode
      dbname: helixcode_prod
    redis:
      host: helixcode-redis
      port: 6379
    workers:
      max_concurrent_tasks: 10
      health_check_interval: 30
```

```
apiVersion: v1
kind: Secret
metadata:
  name: helixcode-secrets
  namespace: helixcode
type: Opaque
data:
  database-password: <base64-encoded-password>
  redis-password: <base64-encoded-password>
  openai-api-key: <base64-encoded-key>
  anthropic-api-key: <base64-encoded-key>
```

PostgreSQL StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: helixcode-postgres
  namespace: helixcode
spec:
  serviceName: helixcode-postgres
  replicas: 1
  selector:
```



```

matchLabels:
  app: helixcode-postgres
template:
  metadata:
    labels:
      app: helixcode-postgres
  spec:
    containers:
      - name: postgres
        image: postgres:15-alpine
        ports:
          - containerPort: 5432
        env:
          - name: POSTGRES_DB
            value: "helixcode_prod"
          - name: POSTGRES_USER
            value: "helixcode"
          - name: POSTGRES_PASSWORD
            valueFrom:
              secretKeyRef:
                name: helixcode-secrets
                key: database-password
        volumeMounts:
          - name: postgres-storage
            mountPath: /var/lib/postgresql/data
    resources:
      requests:
        memory: "512Mi"
        cpu: "250m"
      limits:
        memory: "1Gi"
        cpu: "500m"
    volumeClaimTemplates:
      - metadata:
          name: postgres-storage
          namespace: helixcode
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi

```

Redis Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: helixcode-redis

```

```

namespace: helixcode
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helixcode-redis
  template:
    metadata:
      labels:
        app: helixcode-redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          command: ["redis-server", "--requirepass", "$(REDIS_PASSWORD)"]
          env:
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: helixcode-secrets
                  key: redis-password
      volumeMounts:
        - name: redis-storage
          mountPath: /data
      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"
        limits:
          memory: "512Mi"
          cpu: "200m"
      volumes:
        - name: redis-storage
          emptyDir: {}

```

HelixCode Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: helixcode
  namespace: helixcode
spec:
  replicas: 3
  selector:

```

```
matchLabels:
  app: helixcode
template:
  metadata:
    labels:
      app: helixcode
  spec:
    serviceAccountName: helixcode-sa
    containers:
      - name: helixcode
        image: helixcode/helixcode:latest
        ports:
          - containerPort: 8080
        env:
          - name: HELIXCODE_ENV
            value: "production"
          - name: HELIXCODE_DATABASE_PASSWORD
            valueFrom:
              secretKeyRef:
                name: helixcode-secrets
                key: database-password
          - name: HELIXCODE_REDIS_PASSWORD
            valueFrom:
              secretKeyRef:
                name: helixcode-secrets
                key: redis-password
          - name: OPENAI_API_KEY
            valueFrom:
              secretKeyRef:
                name: helixcode-secrets
                key: openai-api-key
          - name: ANTHROPIC_API_KEY
            valueFrom:
              secretKeyRef:
                name: helixcode-secrets
                key: anthropic-api-key
    volumeMounts:
      - name: config-volume
        mountPath: /app/config
      - name: logs-volume
        mountPath: /app/logs
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
```

```

    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 5
  resources:
    requests:
      memory: "512Mi"
      cpu: "250m"
    limits:
      memory: "1Gi"
      cpu: "500m"
  volumes:
  - name: config-volume
    configMap:
      name: helixcode-config
  - name: logs-volume
    emptyDir: {}

```

Services and Ingress

```

apiVersion: v1
kind: Service
metadata:
  name: helixcode-service
  namespace: helixcode
spec:
  selector:
    app: helixcode
  ports:
  - name: http
    port: 80
    targetPort: 8080
  type: ClusterIP

```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: helixcode-ingress
  namespace: helixcode
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:

```

```

- hosts:
  - helixcode.yourdomain.com
  secretName: helixcode-tls
rules:
- host: helixcode.yourdomain.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: helixcode-service
          port:
            number: 80

```

Cloud Platform Deployment

AWS ECS Fargate

Task Definition

```

{
  "family": "helixcode-task",
  "taskRoleArn": "arn:aws:iam::123456789012:role/helixcode-task-
    role",
  "executionRoleArn": "arn:aws:iam::123456789012:role/helixcode-
    task-execution-role",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "containerDefinitions": [
    {
      "name": "helixcode",
      "image": "helixcode/helixcode:latest",
      "essential": true,
      "portMappings": [
        {
          "containerPort": 8080,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {"name": "HELIXCODE_ENV", "value": "production"},
        {"name": "HELIXCODE_DATABASE_HOST", "value": "${
          DB_HOST}"},
        {"name": "HELIXCODE_DATABASE_PASSWORD", "value": "${
          DB_PASSWORD}"},
      ]
    }
  ],

```

```

    "secrets": [
      {
        "name": "OPENAI_API_KEY",
        "valueFrom": "arn:aws:secretsmanager:us-
east-1:123456789012:secret:helixcode/api-
keys:OPENAI_API_KEY::"
      }
    ],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/helixcode",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "ecs"
      }
    }
  }
}
]
}

```

AWS RDS PostgreSQL

```

# RDS Instance
{
  "DBInstanceIdentifier": "helixcode-prod",
  "DBInstanceClass": "db.t3.medium",
  "Engine": "postgres",
  "EngineVersion": "15.4",
  "DBName": "helixcode_prod",
  "MasterUsername": "helixcode",
  "AllocatedStorage": 100,
  "StorageType": "gp3",
  "BackupRetentionPeriod": 7,
  "MultiAZ": true,
  "VPCSecurityGroups": ["sg-12345678"],
  "DBSubnetGroupName": "helixcode-db-subnet",
  "EnablePerformanceInsights": true,
  "PerformanceInsightsRetentionPeriod": 7
}

```

Google Cloud Run

```

# Cloud Run Service
gcloud run deploy helixcode \
  --image helixcode/helixcode:latest \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \

```

```
--port 8080 \  
--memory 2Gi \  
--cpu 1 \  
--max-instances 10 \  
--min-instances 1 \  
--concurrency 80 \  
--timeout 900 \  
--set-env-vars "HELIXCODE_ENV=production" \  
--set-secrets "DATABASE_PASSWORD=DATABASE_PASSWORD:latest" \  
--set-secrets "REDIS_PASSWORD=REDIS_PASSWORD:latest" \  
--set-secrets "OPENAI_API_KEY=OPENAI_API_KEY:latest"
```

Azure Container Instances

```
{  
  "$schema": "https://schema.management.azure.com/schemas/  
    2019-04-01/deploymentTemplate.json#",  
  "contentVersion": "1.0.0.0",  
  "parameters": {  
    "containerName": {  
      "type": "string",  
      "defaultValue": "helixcode"  
    }  
  },  
  "resources": [  
    {  
      "type": "Microsoft.ContainerInstance/containerGroups",  
      "apiVersion": "2021-09-01",  
      "name": "[parameters('containerName')]",  
      "location": "[resourceGroup().location]",  
      "properties": {  
        "containers": [  
          {  
            "name": "helixcode",  
            "properties": {  
              "image": "helixcode/helixcode:latest",  
              "ports": [  
                {  
                  "port": 8080,  
                  "protocol": "TCP"  
                }  
              ],  
              "environmentVariables": [  
                {  
                  "name": "HELIXCODE_ENV",  
                  "value": "production"  
                }  
              ]  
            }  
          }  
        ]  
      }  
    }  
  ],  
}
```

```

        "resources": {
            "requests": {
                "cpu": 1,
                "memoryInGB": 2
            }
        }
    },
    ],
    "osType": "Linux",
    "ipAddress": {
        "type": "Public",
        "ports": [
            {
                "port": 8080,
                "protocol": "TCP"
            }
        ]
    }
}
]
}

```

Traditional Server Deployment

Ubuntu/Debian Installation

```

#!/bin/bash
# HelixCode Production Installation Script for Ubuntu/Debian

set -e

# Update system
apt update && apt upgrade -y

# Install dependencies
apt install -y curl wget gnupg2 software-properties-common

# Install Go
wget -q https://go.dev/dl/go1.26.3.linux-amd64.tar.gz
rm -rf /usr/local/go && tar -C /usr/local -xzf go1.26.3.linux-
    amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
echo 'export PATH=$PATH:/usr/local/go/bin' >> /etc/profile

# Install PostgreSQL
apt install -y postgresql postgresql-contrib

```



```
systemctl enable postgresql
systemctl start postgresql
```

Install Redis

```
apt install -y redis-server
systemctl enable redis-server
systemctl start redis-server
```

Create database and user

```
sudo -u postgres psql << EOF
CREATE DATABASE helixcode_prod;
CREATE USER helixcode WITH PASSWORD 'secure_password_here';
GRANT ALL PRIVILEGES ON DATABASE helixcode_prod TO helixcode;
EOF
```

Create application user

```
useradd -r -s /bin/false helixcode
mkdir -p /opt/helixcode
chown helixcode:helixcode /opt/helixcode
```

Download and install HelixCode

```
cd /tmp
wget https://github.com/your-org/helixcode/releases/latest/
      download/helixcode-linux-amd64.tar.gz
tar -xzf helixcode-linux-amd64.tar.gz
mv helixcode /usr/local/bin/
chmod +x /usr/local/bin/helixcode
```

Create configuration

```
mkdir -p /etc/helixcode
cat > /etc/helixcode/config.yaml << EOF
server:
  host: 0.0.0.0
  port: 8080
  tls_enabled: true
  tls_cert_file: /etc/helixcode/ssl/cert.pem
  tls_key_file: /etc/helixcode/ssl/key.pem
```

database:

```
  host: localhost
  port: 5432
  user: helixcode
  password: secure_password_here
  dbname: helixcode_prod
  sslmode: require
```

redis:

```
  host: localhost
```

```
port: 6379
password: secure_redis_password_here

llm:
  default_provider: openai
  max_tokens: 4096
  temperature: 0.7

workers:
  max_concurrent_tasks: 10
  health_check_interval: 30
  ssh_key_path: /etc/helixcode/ssh/id_rsa

logging:
  level: info
  format: json
  output: /var/log/helixcode/helixcode.log

monitoring:
  enabled: true
  metrics_port: 9090
EOF

# Setup directories
mkdir -p /var/log/helixcode /etc/helixcode/ssl /etc/helixcode/ssh
chown -R helixcode:helixcode /var/log/helixcode /etc/helixcode

# Generate SSH keys for worker communication
ssh-keygen -t rsa -b 4096 -f /etc/helixcode/ssh/id_rsa -N ""
chown helixcode:helixcode /etc/helixcode/ssh/id_rsa*

# Create systemd service
cat > /etc/systemd/system/helixcode.service << EOF
[Unit]
Description=HelixCode AI Development Platform
After=network.target postgresql.service redis-server.service
Requires=postgresql.service redis-server.service

[Service]
Type=simple
User=helixcode
Group=helixcode
ExecStart=/usr/local/bin/helixcode server --config /etc/
        helixcode/config.yaml
Restart=always
RestartSec=5
LimitNOFILE=65536

# Security settings
```

```
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ReadWritePaths=/var/log/helixcode /tmp
ProtectHome=yes
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
# Create logrotate configuration
cat > /etc/logrotate.d/helixcode << EOF
/var/log/helixcode/*.log {
    daily
    missingok
    rotate 52
    compress
    delaycompress
    notifempty
    create 644 helixcode helixcode
    postrotate
        systemctl reload helixcode
    endsript
}
EOF
```

```
# Setup firewall
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow 2222/tcp # SSH for workers
ufw --force enable
```

```
# Start services
systemctl daemon-reload
systemctl enable postgresql redis-server helixcode
systemctl start postgresql redis-server helixcode
```

```
# Setup SSL certificate (Let's Encrypt example)
apt install -y certbot
certbot certonly --standalone -d your-domain.com
ln -s /etc/letsencrypt/live/your-domain.com/fullchain.pem /etc/
    helixcode/ssl/cert.pem
ln -s /etc/letsencrypt/live/your-domain.com/privkey.pem /etc/
    helixcode/ssl/key.pem
```

```
# Setup nginx reverse proxy
apt install -y nginx
cat > /etc/nginx/sites-available/helixcode << EOF
```

```

server {
    listen 80;
    server_name your-domain.com;
    return 301 https://\${server_name}\${request_uri};
}

server {
    listen 443 ssl http2;
    server_name your-domain.com;

    ssl_certificate /etc/letsencrypt/live/your-domain.com/
        fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/
        privkey.pem;

    # SSL security settings
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-
        SHA512:ECDHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers off;

    # Security headers
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";
    add_header Strict-Transport-Security "max-age=63072000;
        includeSubDomains; preload";

    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host \${host};
        proxy_set_header X-Real-IP \${remote_addr};
        proxy_set_header X-Forwarded-For \
            \${proxy_add_x_forwarded_for};
        proxy_set_header X-Forwarded-Proto \${scheme};

        # Timeout settings
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Static file caching
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}

```

EOF

```
ln -s /etc/nginx/sites-available/helixcode /etc/nginx/sites-enabled/
rm /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx

echo "HelixCode installation completed!"
echo "Access your instance at: https://your-domain.com"
echo "Check logs with: journalctl -u helixcode -f"
```

High Availability Setup

Load Balancer Configuration

```
# /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;

events {
    worker_connections 1024;
    use epoll;
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Logging
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    # Performance
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    client_max_body_size 100M;

    # Gzip compression
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_proxied any;
    gzip_comp_level 6;
```

```

gzip_types
    text/plain
    text/css
    text/xml
    text/javascript
    application/json
    application/javascript
    application/xml+rss
    application/atom+xml
    image/svg+xml;

# Upstream backend servers
upstream helixcode_backend {
    least_conn;
    server helixcode-01:8080 max_fails=3 fail_timeout=30s;
    server helixcode-02:8080 max_fails=3 fail_timeout=30s;
    server helixcode-03:8080 max_fails=3 fail_timeout=30s;
}

server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name your-domain.com;

    ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # SSL settings
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers off;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    # Security headers
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Strict-Transport-Security "max-age=63072000; includeSubdomains;";
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Main application
    location / {
        proxy_pass http://helixcode_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

```

    # WebSocket support
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";

    # Timeouts
    proxy_connect_timeout 60s;
    proxy_send_timeout 60s;
    proxy_read_timeout 60s;

    # Health checks
    health_check interval=10s fails=3 passes=2;
}

# Static file caching
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)
    proxy_pass http://helixcode_backend;
    expires 1y;
    add_header Cache-Control "public, immutable";
}

# API rate limiting
location /api/ {
    limit_req zone=api burst=100 nodelay;
    proxy_pass http://helixcode_backend;
}
}

```

```

# Rate limiting zones
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=auth:10m rate=5r/m;

```

Database Replication Setup

```

-- Primary database setup
-- postgresql.conf
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
wal_keep_size = 64MB

-- pg_hba.conf
host replication replicator 192.168.1.0/24 md5

-- Create replication user
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD
    'replication_password';

-- Secondary database setup
-- recovery.conf

```

```
primary_conninfo = 'host=primary_host port=5432 user=replicator
                    password=replication_password'
restore_command = 'cp /var/lib/postgresql/archive/%f %p'
recovery_target_timeline = 'latest'
```

Redis Cluster Setup

```
# redis-cluster.yaml
version: '3.8'

services:
  redis-1:
    image: redis:7-alpine
    command: redis-server /etc/redis/redis.conf
    volumes:
      - ./redis-1.conf:/etc/redis/redis.conf
      - redis-1-data:/data
    networks:
      - redis-cluster

  redis-2:
    image: redis:7-alpine
    command: redis-server /etc/redis/redis.conf
    volumes:
      - ./redis-2.conf:/etc/redis/redis.conf
      - redis-2-data:/data
    depends_on:
      - redis-1
    networks:
      - redis-cluster

  redis-3:
    image: redis:7-alpine
    command: redis-server /etc/redis/redis.conf
    volumes:
      - ./redis-3.conf:/etc/redis/redis.conf
      - redis-3-data:/data
    depends_on:
      - redis-2
    networks:
      - redis-cluster

networks:
  redis-cluster:
    driver: bridge

volumes:
  redis-1-data:
```



```
redis-2-data:  
redis-3-data:
```

Monitoring and Observability

Prometheus Configuration

```
# prometheus.yml  
global:  
  scrape_interval: 15s  
  evaluation_interval: 15s  
  
rule_files:  
  - "alert_rules.yml"  
  
alerting:  
  alertmanagers:  
    - static_configs:  
      - targets:  
        - alertmanager:9093  
  
scrape_configs:  
  - job_name: 'helixcode'  
    static_configs:  
      - targets: ['helixcode:8080']  
    scrape_interval: 5s  
    metrics_path: '/metrics'  
  
  - job_name: 'postgres'  
    static_configs:  
      - targets: ['postgres-exporter:9187']  
  
  - job_name: 'redis'  
    static_configs:  
      - targets: ['redis-exporter:9121']  
  
  - job_name: 'node'  
    static_configs:  
      - targets: ['node-exporter:9100']
```

Grafana Dashboards

```
{  
  "dashboard": {  
    "title": "HelixCode Overview",  
    "tags": ["helixcode", "overview"],  
    "timezone": "browser",  
    "panels": [  

```

```

{
  "title": "Request Rate",
  "type": "graph",
  "targets": [
    {
      "expr": "rate(http_requests_total[5m])",
      "legendFormat": "Requests/sec"
    }
  ]
},
{
  "title": "Response Time",
  "type": "graph",
  "targets": [
    {
      "expr": "histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m]))",
      "legendFormat": "95th percentile"
    }
  ]
},
{
  "title": "Active Tasks",
  "type": "stat",
  "targets": [
    {
      "expr": "helixcode_tasks_active",
      "legendFormat": "Active Tasks"
    }
  ]
},
{
  "title": "LLM Token Usage",
  "type": "graph",
  "targets": [
    {
      "expr": "rate(helixcode_llm_tokens_total[5m])",
      "legendFormat": "Tokens/min"
    }
  ]
}
]
}

```

Backup and Recovery

Automated Backup Script

```
#!/bin/bash
# HelixCode Automated Backup Script

set -e

BACKUP_DIR="/opt/helixcode/backups"
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
BACKUP_NAME="helixcode_backup_${TIMESTAMP}"

# Create backup directory
mkdir -p "${BACKUP_DIR}/${BACKUP_NAME}"

# Database backup
echo "Backing up PostgreSQL database..."
pg_dump -h localhost -U helixcode -d helixcode_prod | gzip > "${BACKUP_DIR}/${BACKUP_NAME}/database.sql.gz"

# Redis backup
echo "Backing up Redis data..."
redis-cli -a "${REDIS_PASSWORD}" --rdb "${BACKUP_DIR}/${BACKUP_NAME}/redis.rdb"

# Configuration backup
echo "Backing up configuration..."
cp -r /etc/helixcode "${BACKUP_DIR}/${BACKUP_NAME}/config"

# File storage backup
echo "Backing up file storage..."
tar -czf "${BACKUP_DIR}/${BACKUP_NAME}/files.tar.gz" -C /opt/helixcode data/

# Create backup manifest
cat > "${BACKUP_DIR}/${BACKUP_NAME}/manifest.json" << EOF
{
  "backup_name": "${BACKUP_NAME}",
  "timestamp": "${TIMESTAMP}",
  "version": "$(/usr/local/bin/helixcode version)",
  "components": [
    "database",
    "redis",
    "config",
    "files"
  ],
  "compression": "gzip",
```

```

    "size_bytes": $(du -sb "${BACKUP_DIR}/${BACKUP_NAME}" | cut
        -f1)
}
EOF

# Cleanup old backups (keep last 30 days)
find "${BACKUP_DIR}" -name "helixcode_backup_*" -type d -mtime
    +30 -exec rm -rf {} +

echo "Backup completed: ${BACKUP_NAME}"

```

Recovery Procedure

```

#!/bin/bash
# HelixCode Recovery Script

set -e

BACKUP_NAME="$1"
BACKUP_DIR="/opt/helixcode/backups"

if [ -z "$BACKUP_NAME" ]; then
    echo "Usage: $0 <backup_name>"
    exit 1
fi

if [ ! -d "${BACKUP_DIR}/${BACKUP_NAME}" ]; then
    echo "Backup ${BACKUP_NAME} not found"
    exit 1
fi

echo "Starting recovery from backup: ${BACKUP_NAME}"

# Stop services
systemctl stop helixcode

# Restore database
echo "Restoring PostgreSQL database..."
gunzip -c "${BACKUP_DIR}/${BACKUP_NAME}/database.sql.gz" | psql
    -h localhost -U helixcode -d helixcode_prod

# Restore Redis
echo "Restoring Redis data..."
redis-cli -a "${REDIS_PASSWORD}" shutdown || true
cp "${BACKUP_DIR}/${BACKUP_NAME}/redis.rdb" /var/lib/redis/
    dump.rdb
chown redis:redis /var/lib/redis/dump.rdb
systemctl start redis-server

```

```
# Restore configuration
echo "Restoring configuration..."
cp -r "${BACKUP_DIR}/${BACKUP_NAME}/config" /etc/

# Restore files
echo "Restoring file storage..."
tar -xzf "${BACKUP_DIR}/${BACKUP_NAME}/files.tar.gz" -C /opt/
    helixcode

# Start services
systemctl start helixcode

echo "Recovery completed successfully"
```

Security Hardening

SSL/TLS Configuration

```
# Strong SSL configuration
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512:ECDHE-RS
ssl_prefer_server_ciphers off;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 10m;

# HSTS
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains;

# Security headers
add_header X-Frame-Options DENY always;
add_header X-Content-Type-Options nosniff always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'
```

System Hardening

```
#!/bin/bash
# System hardening script

# Disable unused services
systemctl disable avahi-daemon
systemctl disable cups
systemctl disable bluetooth

# Configure SSH
cat >> /etc/ssh/sshd_config << EOF
PermitRootLogin no
PermitEmptyPasswords no
PasswordAuthentication no
```

```
ChallengeResponseAuthentication no
UsePAM yes
X11Forwarding no
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
EOF
```

```
systemctl reload sshd
```

```
# Configure firewall
```

```
ufw default deny incoming
ufw default allow outgoing
ufw allow ssh
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable
```

```
# Disable core dumps
```

```
echo '* hard core 0' >> /etc/security/limits.conf
echo '* soft core 0' >> /etc/security/limits.conf
echo 'fs.suid_dumpable = 0' >> /etc/sysctl.conf
sysctl -p
```

```
# Configure audit logging
```

```
apt install -y auditd
cat > /etc/audit/rules.d/helixcode.rules << EOF
# HelixCode audit rules
-w /usr/local/bin/helixcode -p x
-w /etc/helixcode -p wa
-w /opt/helixcode -p wa
EOF
```

```
systemctl enable auditd
systemctl start auditd
```

Performance Tuning

Application Configuration

```
# config/production.yaml
server:
  host: 0.0.0.0
  port: 8080
  read_timeout: 30s
  write_timeout: 30s
  max_header_bytes: 1048576
  max_concurrent_connections: 1000
```

```
database:
  host: localhost
  port: 5432
  user: helixcode
  password: ${DB_PASSWORD}
  dbname: helixcode_prod
  sslmode: require
  max_open_conns: 25
  max_idle_conns: 5
  conn_max_lifetime: 5m

redis:
  host: localhost
  port: 6379
  password: ${REDIS_PASSWORD}
  db: 0
  pool_size: 10
  min_idle_conns: 5
  conn_max_lifetime: 30m

llm:
  default_provider: openai
  max_tokens: 4096
  temperature: 0.7
  timeout: 60s
  max_retries: 3
  rate_limit_requests: 100
  rate_limit_window: 60s

workers:
  max_concurrent_tasks: 20
  health_check_interval: 15s
  task_timeout: 30m
  worker_selection_strategy: load_balanced

cache:
  enabled: true
  ttl: 300s
  max_memory: 512MB
  eviction_policy: lru

logging:
  level: info
  format: json
  output: /var/log/helixcode/helixcode.log
  max_size: 100MB
  max_backups: 5
```

```
max_age: 30
compress: true
```

System Tuning

```
#!/bin/bash
# System performance tuning
```

```
# Kernel parameters
cat >> /etc/sysctl.conf << EOF
# Network tuning
net.core.somaxconn = 65536
net.ipv4.tcp_max_syn_backlog = 65536
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
```

```
# Memory management
vm.swappiness = 10
vm.dirty_ratio = 60
vm.dirty_background_ratio = 2
```

```
# File system
fs.file-max = 2097152
fs.inotify.max_user_watches = 524288
EOF
```

```
sysctl -p
```

```
# PostgreSQL tuning
cat >> /etc/postgresql/15/main/postgresql.conf << EOF
```

```
# Memory
shared_buffers = 256MB
effective_cache_size = 1GB
work_mem = 4MB
maintenance_work_mem = 64MB
```

```
# Connections
max_connections = 100
```

```
# Logging
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h'
log_statement = 'ddl'
log_duration = on
log_lock_waits = on
EOF
```

```
systemctl restart postgresql
```



```
# Redis tuning
cat >> /etc/redis/redis.conf << EOF
# Memory management
maxmemory 256mb
maxmemory-policy allkeys-lru

# Persistence
save 900 1
save 300 10
save 60 10000

# Performance
tcp-keepalive 300
timeout 0
EOF
```

```
systemctl restart redis-server
```

Troubleshooting

Common Issues

Database Connection Issues

Symptoms: - Application fails to start - Error: “connection refused” - Slow response times

Solutions:

```
# Check PostgreSQL status
systemctl status postgresql

# Check connection
psql -h localhost -U helixcode -d helixcode_prod

# Check logs
tail -f /var/log/postgresql/postgresql-15-main.log

# Reset password if needed
sudo -u postgres psql
ALTER USER helixcode PASSWORD 'new_password';
```

Redis Connection Issues

Symptoms: - Session errors - Caching not working - Slow performance

Solutions:

```
# Check Redis status
systemctl status redis-server

# Test connection
redis-cli -a ${REDIS_PASSWORD} ping

# Check memory usage
redis-cli -a ${REDIS_PASSWORD} info memory

# Clear cache if needed
redis-cli -a ${REDIS_PASSWORD} FLUSHALL
```

Worker Connection Issues

Symptoms: - Tasks stuck in pending - Worker health checks failing - SSH connection errors

Solutions:

```
# Check SSH key permissions
ls -la /etc/helixcode/ssh/

# Test SSH connection
ssh -i /etc/helixcode/ssh/id_rsa -o StrictHostKeyChecking=no
worker-host

# Check worker logs
tail -f /var/log/helixcode/worker.log

# Restart worker service
systemctl restart helixcode-worker
```

High Memory Usage

Symptoms: - Application consuming excessive memory - Out of memory errors - System slowdown

Solutions:

```
# Check memory usage
ps aux --sort=-%mem | head

# Check Go memory stats
curl http://localhost:8080/debug/pprof/heap > heap.prof
go tool pprof heap.prof

# Adjust memory limits
# In config.yaml
memory:
  max_heap_size: 512MB
  gc_percent: 80
```

```
# Restart application
systemctl restart helixcode
```

Slow API Response Times

Symptoms: - API calls taking too long - Timeout errors - Poor user experience

Solutions:

```
# Check system resources
top
iostat -x 1
free -h

# Check database performance
psql -d helixcode_prod -c "SELECT * FROM pg_stat_activity;"

# Check Redis performance
redis-cli -a ${REDIS_PASSWORD} info stats

# Enable query logging
# In config.yaml
database:
  log_queries: true
  slow_query_threshold: 1s

# Check application metrics
curl http://localhost:8080/metrics
```

Log Analysis

```
# Search for errors
grep "ERROR" /var/log/helixcode/helixcode.log | tail -20

# Check for specific error patterns
grep "connection refused" /var/log/helixcode/helixcode.log

# Analyze slow queries
grep "SLOW QUERY" /var/log/postgresql/postgresql-15-main.log

# Monitor real-time logs
tail -f /var/log/helixcode/helixcode.log | grep -E "(ERROR|WARN)"
```

Health Checks

```
# Application health
curl -f http://localhost:8080/health
```

```
# Database health
psql -h localhost -U helixcode -d helixcode_prod -c "SELECT 1;"

# Redis health
redis-cli -a ${REDIS_PASSWORD} ping

# Worker health
curl -f http://localhost:8080/api/v1/workers/health

# System resources
df -h
free -h
uptime
```

Emergency Recovery

```
# Quick restart
systemctl restart helixcode

# Force restart if hanging
systemctl kill helixcode
systemctl start helixcode

# Rollback to previous version
cd /opt/helixcode
cp backup/helixcode.previous /usr/local/bin/helixcode
systemctl restart helixcode

# Emergency database recovery
# Stop application first
systemctl stop helixcode

# Recover from backup
/opt/helixcode/scripts/recovery.sh latest

# Start services
systemctl start postgresql redis-server helixcode
```

This deployment guide covers production-ready setups for HelixCode. For additional support, visit our documentation site or contact enterprise support. docs/DEPLOYMENT_GUIDE.md ## Sources verified Sources verified 2026-05-29: <https://go.dev/dl/> (go1.26.3 latest stable Go; 1.24 past support) ; project go.mod (root go 1.25.2, inner go 1.26) + CLAUDE.md §3.1 (PostgreSQL 15+).